



NLP 입문



NLP란?

자연어를 컴퓨터가 이해하고 처리하는 과정

반드시 AI에 한정된 것은 아니지만 일반적으로 NLP에는
딥러닝을 상정함

자연어란? 자연상에서 발생한 사람의 말

↔ 인공어(Artificial Language)



NLP란?

언어 형태(representation)로 표현 가능한 모든 의미를
모델링하고 변환하는 문제



자연어의 특징

- 맥락 의존성 : 같은 문장이 맥락에 따라 의미가 변화
- 불완전성 : 규범을 충족하지 않을 수 있음
- 다양성 : 언어, 방언, 은어 등 형식과 표현이 다양



다양한 NLP task

- 품사 태깅(POS Tagging)
- 개체명 인식(NER)
- 구문 분석(Parsing)
- 질의 응답(QA)
- 번역(Machine Translation)
- 요약(Summarization)



Transformer 이후의 변화

- 동일 모델 구조(Transformer) 사용
- 기존 문제들을 텍스트 생성 문제로 통일
즉, 다음 토큰 예측으로 통일



자연어는 왜 생성 문제인가?

- 언어는 단어-단어-단어로 이어지는 생성 과정
- 의미가 분포적으로 표현됨
- task들이 사실 동일 문제군
- NLG는 NLU가 전제되어있음



생성으로 처리했을 때 문제점

- 출력 구조 불안정
- 정밀도가 아닌 “그럴듯함”
- 제약 조건 적용
- 효율성 및 비용
- 평가 가능성



NLP task를 이해해야하는 이유

- 생성 모델도 각각의 task를 내부적으로 수행(내제화)
- 무엇을 잘하는지 못하는지 판단은 task 단위 수행
- 실제 시스템은 완전한 end-to-end가 드뭄
- 각 task별 다른 요구사항



모델 출력 설계



인공지능 파이프라인

1. 데이터 수집
2. 데이터 정제
3. 표현 변환
4. 모델 입력
5. 텐서 계산
6. 출력 후 디코딩



NLP 파이프라인

1. Tokenizing
2. Encoding
3. Vectorization
4. Inference
5. Decoding



Tokenizing

문장을 컴퓨터가 처리할 수 있는 작은 단위로 분리

현재는 메모리를 절약하면서도 의미를 보존할 수 있는
subword 단위를 주로 사용



Encoding

각 토큰을 정수 ID로 매핑하는 과정

정수 ID 값이 무한정 있을 수 없기 때문에 어떤 토큰을
등록할지 결정 필요



Vectorization

각 정수 인덱스 자체는 순서 이상의 의미를 가지지 못하기
때문에 벡터 공간상에 분포시켜 의미를 가진 데이터로 변환
각 단어 간 의미적, 패턴적 관계를 반영하게 됨



Inference

모델(RNN, Transformer 등)이 임베딩 벡터를 받아
문맥적 특징을 학습

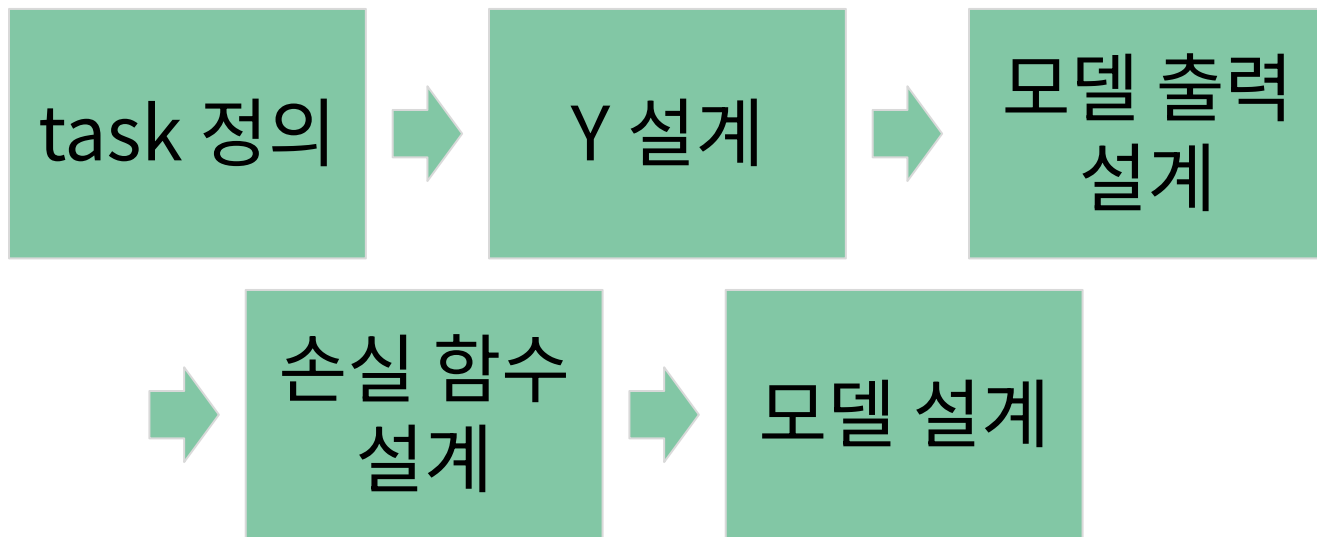
단, 임베딩 벡터 변환을 모델 추론의 과정에 포함할 수 있음



Decoding

추론 결과를 사람이 해석할 수 있도록 변환하는 과정
분류라면 각 class 별 확률, 생성이라면 자연어 단어
등으로 변환

설계 순서





모델 출력값 설계

- Classification : Discrete Label
- Regression : Continuous Value



출력 값 설계 예시

Summarization

- 원문의 고유 명사가 보존되어야함
- 길이가 원문보다 짧아야함

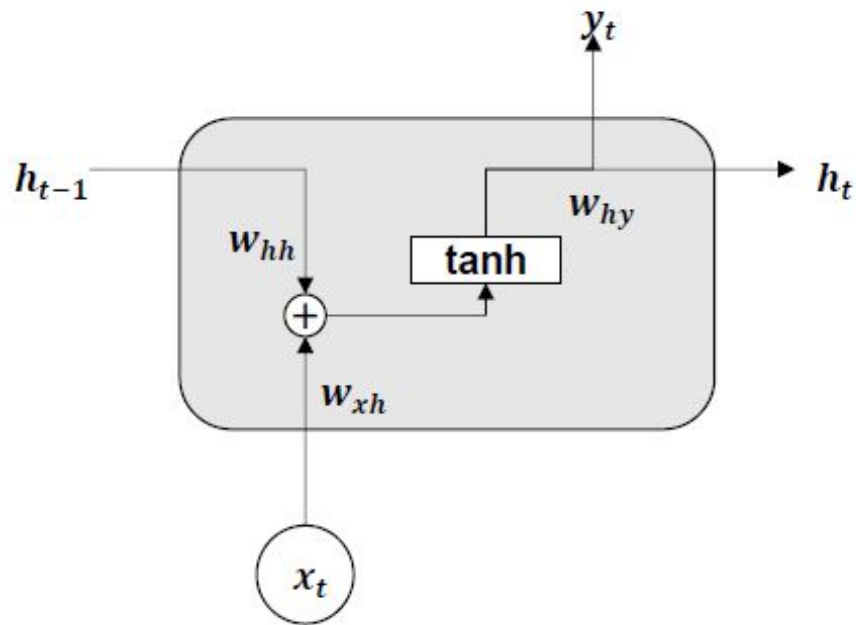
...

-> 다음 단어를 예측 반복 -> 각 단어별 확률

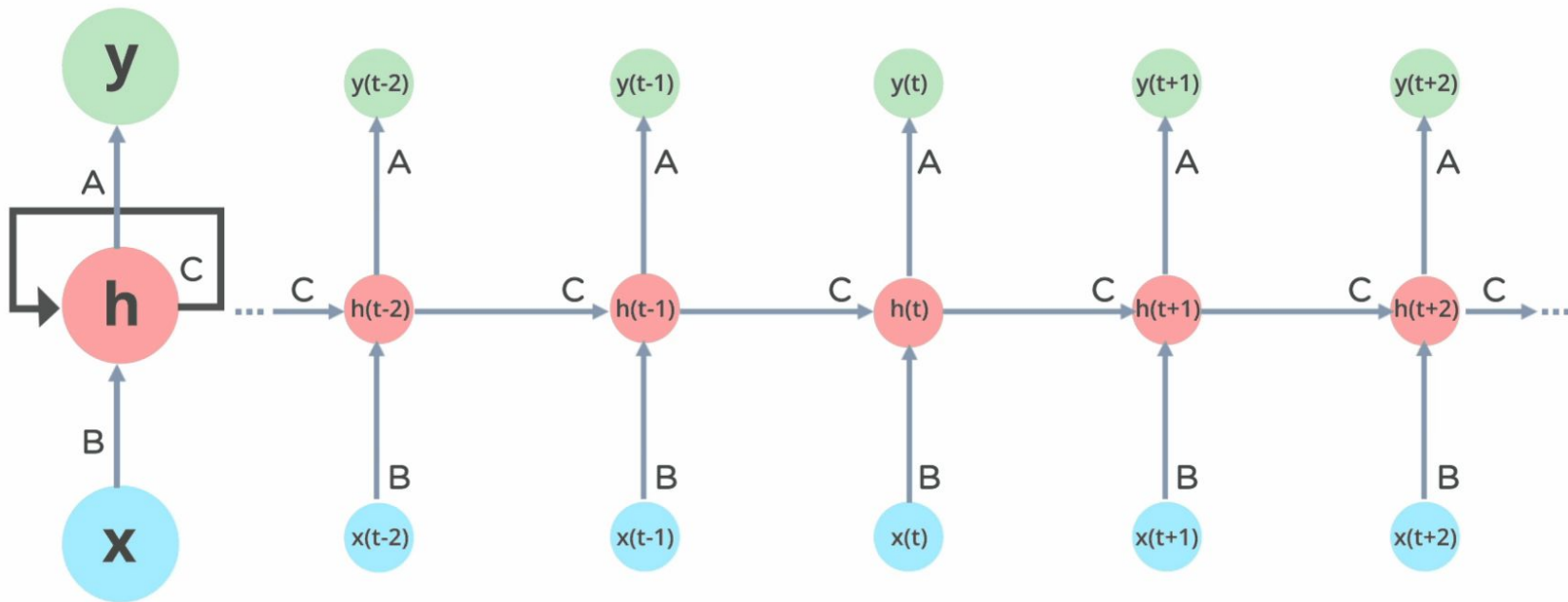


RNN과 텍스트 생성

RNN의 구조



RNN의 동작



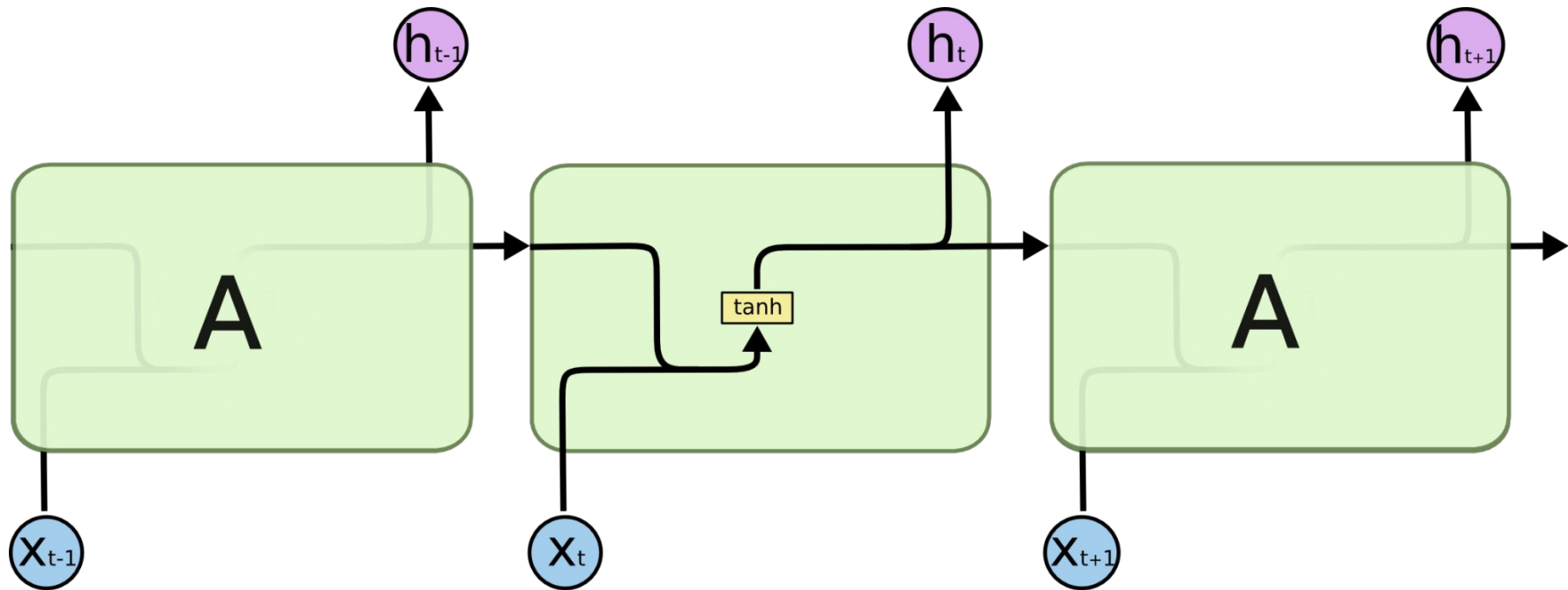
Next Token Prediction

모든 생성은 결국 vocab 위의 확률 분포를 예측하는 문제



$$\hat{y} = P(w_t \mid context) = \text{softmax}(Wx + b)$$

RNN의 구조

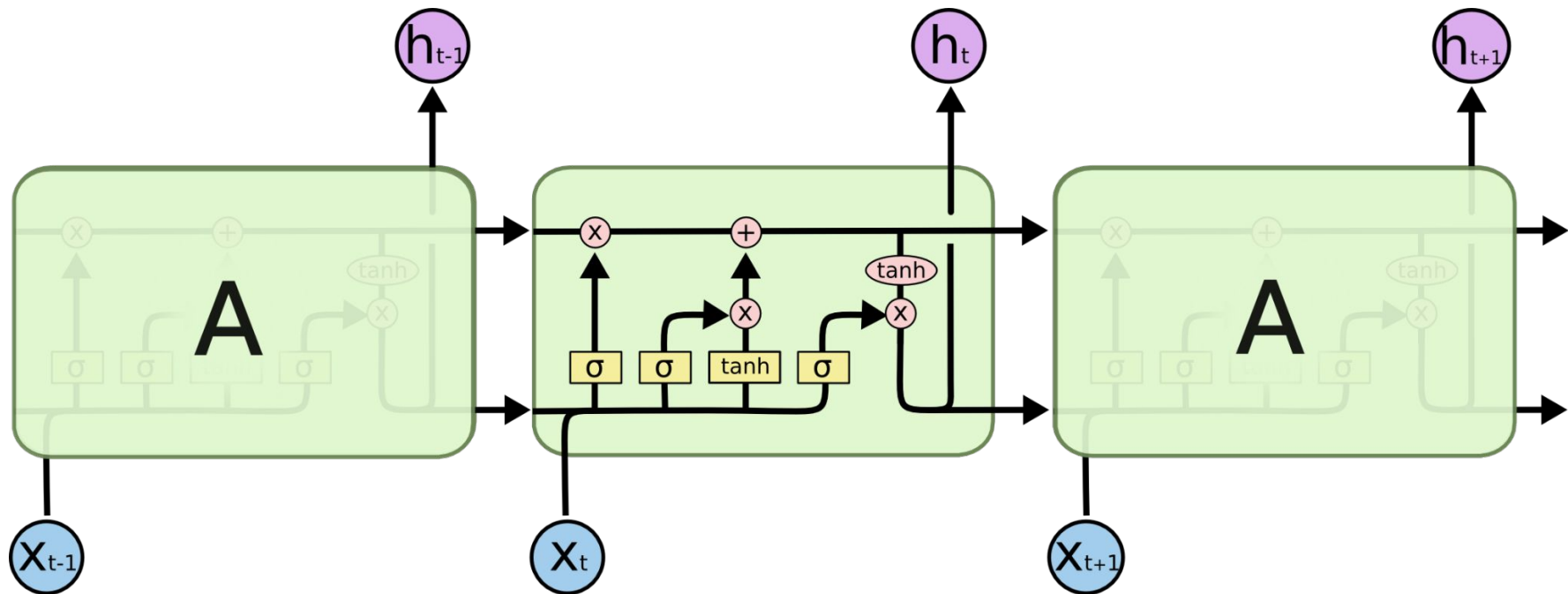




수식으로 보는 RNN

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$$

LSTM의 구조구조





수식으로 보는 LSTM

word vector : x_t

cell state : c_{t-1}

hidden state : h_{t-1}

cell state는 장기 기억,

hidden state는 단기 기억

각 제어는 단기 기억의 영향을
받음(gate)



수식으로 보는 LSTM

forget gate : $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

input gate : $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

후보 cell state : $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$

최종 cell state : $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

output gate : $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

hidden state : $h_t = o_t \odot \tanh(C_t)$